Task D — Validation and Algorithm Analysis

KEY:

- [ADDED] or [FIX] are corrections made by Claude Opus 5.5 Medium.
- Text that is struck-through or non-bolded are the original developer claims and understanding.

1. Algorithm Design

Task A — Selecting the pair of points with the minimal distance

Divide: ~~Sort the main list into two lists, by x and y. Then divide the list of x values in half.~~ **[FIX: Sort the points once by x and once by y. Split them in half by x at a dividing line, keeping each half's y order, until a piece has 3 or fewer points.]**

Conquer: ~~For each half, compute the distances for both.~~ **[FIX: Solve each half recursively, comparing every pair at 3 or fewer points. d is the smaller of the two answers.]**

Combine: ~~Combine both halves, compute the distances for pairs that cross the threshold, select the pair with the minimal distance. Anything outside of this minimal distance in the x and y direction are discarded. Pass this value up.~~ **[FIX: Check only points within d of the line (x-strip), in y order, each against the points above it until one is d or more higher (y-strip). Pass the closest pair up.]**

Essentially, Task A is given a list of ~~pairs~~ **[FIX: points]** that have x and y coordinates. We want to find the two ~~pairs~~ **[FIX: points]** that have the minimal distance between each other from the list. X-Strip and Y-Strip techniques provided by the ~~maximal~~ **[FIX: current minimal]** distance d ~~helps~~ **[FIX: help]** us cut down the pairs needed to be checked as we ascend up the levels.

Task B — Counting Inversions

Divide: ~~Convert each value to a ranking system, for both the actual and planned lists.~~ **[FIX: Replace each ID in actual with its position in planned.]** Continously divide the ~~lists~~ **[FIX: rank list]** until only one value remains, as that will be the base case, no inversions for a singular value.

Conquer: ~~For both halves, recursively count the inversions for left, right, and across the divider. Keep a running total.~~ **[FIX: Recursively count the inversions inside each half; each call also returns its half sorted.]**

Combine: [ADDED: Merge the sorted halves with two pointers. Each time a right value is taken first, add the number of values still in the left half. Total = left + right + across.] Pass this value and the resulting merged list upward, as that will be one of the halves the next level needs for its own ~~"conquering"~~ **[FIX: merge]**.

Essentially, Task B is provided a predicted plan of deliveries, and the actual resulting plan of deliveries, which may or may not match exactly. The goal is to find the number of inverted pairs, i.e. ~~(D1, D3) go in the wrong order.~~ **[FIX: D3 is delivered before D1, though D1 was planned first.]** We split to their simplest forms, as a case of one value is the easiest to resolve. By resolving that piece and its corresponding half, we move up, and can now solve the next pieces, and the ones that succeed them.

Task C — Finding the kth smallest value

Divide: Using the whole array, split into groups of 5 elements each. Then, ~~recursively retrieve the median for each piece.~~ **[FIX: sort each group and take its median.]** Obtain the median of those medians recursively as well. **[ADDED: That is the pivot.]**

Conquer: Provided the ~~median~~ **[FIX: pivot]**, ~~sort~~ **[FIX: place]** the values into three categories of being less than pivot, same as pivot, greater than pivot. Based on the value of k, select lesser, same, or greater. If same is selected, ~~the pivot value is k.~~ **[FIX: the pivot is the answer.]** If lesser is selected, k value does not change. If greater is selected, k is updated to be -> `k = k - (len of lesser) - (len of same)`. The algorithm repeats on the chosen partition, others discarded.

Combine: No combine step, partitions are discarded on selection, so the resulting list gets smaller and smaller, not bigger and bigger as it goes on. When a list chosen has 5 or less values, kth is guaranteed to be present. **[ADDED: Sort it and take the kth value.]**

Essentially, Task C is about finding the ~~smallest value at a specific index.~~ **[FIX: kth smallest value.]** The ~~index~~ **[FIX: k value]** has to update based on the partition chosen when we ~~filter values by medians~~ **[FIX: split values around the pivot]**. This helps eliminate branches, which is very similar to binary search. Interestingly, it does not have the combine step, ~~like~~ **[FIX: unlike]** task A and B, because it is an ever shrinking list of values through partition discarding.

2. Time and Space Complexity

A-Time

O(n log n), as requested by the assignment. ~~The pseudocode used is as follows:~~

Examine the two sorts at the beginning, that takes n log n each, ~~given the sorting algorithm is finished.~~ **[FIX: and they happen only once.]** As it stands, we are at `time = n log n + n log n`. Next, we have to scan "walk pointsByY once", so thats linear. **[ADDED: The strip check is also linear (see below).]** We do this on log n levels (tree). So that's n log n. So we're still maintaining: `time = n log n + n log n + n log n`, or simplified to just n log n. All other actions taken are smaller in complexity than n log n or equal to, and the resulting time complexity is simply **O(n log n)**.

A-Space

~~The only extra memory utilized for the algorithm is when y has to be scanned, which requires n memory. Sorts are in-place, they don't use extra memory.~~ **[FIX: Two sorted copies of the input, plus per-level lists that add up to under 2n along one recursion path.]** Memory is **O(n)**.

Why does combine not account all pairs

This is because the current minimal distance calculated from the prior level fits a "range" on what pairs are acceptable going forward. ~~So pairs of points outside the range classified will be ignored, or are a "discarded partition".~~ **[FIX: Only points within d of the line can form a closer crossing pair. Inside the strip, neither half has a pair closer than d, so at most 7 later strip points can be within d above any point. The check stops there, so the strip costs linear time instead of every pair.]**

B-Time and B-Space

~~Given the above pseudocode, we shall analyze the running time.~~ Here are some PoIs in the code that may affect the runtime:

- Scanning to build the ranks; that's linear complexity. O(n)
- sortAndCount is a unique merge sort implementation, so O(n log n) **[ADDED: log n levels, n work each.]**

So currently we have a time complexity of about: O(n log n + n), or **O(n log n)** simplified.

As for space, we need to store the ranking system as its own new list, which is an extra usage of n items. **[ADDED: The position map and merge buffer are n each too, so O(n).]** The depth of the tree may be considerable as well, as O(log n).

C-Time and C-Space

~~Given the above pseudocode, we will analyze the PoIs that may affect runtime. Here are some:~~

- medianOfMedians -> this function call sorts and gets the median of each piece. ~~Approximately logarithmic growth~~ **[FIX: Sorting 5 values is constant work]** for each piece, over n/5 times. So, that's about n complexity summed up.
- **[ADDED: The split is one pass (n). The two recursive calls are on n/5 medians and at most 7n/10 kept values: `T(n) <= T(n/5) + T(7n/10) + n`. Since 1/5 + 7/10 < 1, this is O(n).]**
- ~~For each partition created over the runtime, that's about n/5 space,~~ **[FIX: Medians are n/5 and the kept partition at most 7n/10;]** ~~which~~ **[FIX: these]** will still sum up to n usage in memory, even if the arrays get smaller over each recursive call.

Why is a good pivot important

- The pivot, or the median of the medians list, is important because its essentially the closest thing to a binary search "middle" value over unsorted data, and we are not allowed to sort the whole list.
- This great pivot lets us decide: "ok, where's our kth value, is it lesser than this, greater, or is it this?"
- And because our pivot is good, we can have confidence in discarding certain partitions as we guarantee that our kth value is not in the others.
- ~~Had the pivot be incorrect, we'd likely erroneously discard the correct kth value, caused by splitting incorrectly. It may also create more work for us.~~ **[FIX: Any pivot gives the right answer; a bad one only discards less, making the worst case $O(n^2)$. Median of medians guarantees about 3/10 of the values on each side of the pivot, so each round discards at least 30%.]**

3. Correctness Testing

In my implementation, ~~suite/ReferenceChecks is where the brute-force approaches are tested.~~ **[FIX: `ReferenceChecks` (A), `InversionRefCheck` (B), and `SelectionRefCheck` (C) each compare the algorithm with a brute-force method on 500 random small inputs (seed 413). With the required and extra cases, all 37 suite checks pass.]**

- **Task A:** For task A, the specific choice here was to check every single point against all other points to guarantee that the minimum would be found. These inputs were randomly generated using a fixed seed. For the A-ref to pass, the resulting d value would have to match. **[ADDED: Edge case: `A7-large-coords`, coordinates at ±1,000,000,000 with no overflow.]**
- **Task B:** For task B, two nested loops was utilized as per the assignment details. ~~Particularly, the values from actual, and planned, are compared together. When a mismatch is found, a loop must check other conflicting values for those combinations.~~ **[FIX: Every pair i < j in actual is counted when its planned order is reversed.] [ADDED: Edge case: `B5-large`, 100,000 reversed IDs give 4,999,950,000, beyond int range.]**
- **Task C:** For task C, the whole list is sorted. Then ~~the the list is parsed, for the "kth" value.~~ **[FIX: index k - 1 is read.] [ADDED: Edge case: `C8-many-dupes`, 100,000 values from 0–9.]**

4. Performance

[ADDED: all of section 4.]

Median of 11 timed runs after 5 warm-ups, seed 413, OpenJDK 27 on a 12-core Linux machine (`edu.pioneer.Benchmark`). Ratio = time ÷ time at the previous size.

n	Task A (ms)	ratio	Task B (ms)	ratio	Task C (ms)	ratio
100	0.303	-	0.029	-	0.044	-
1,000	1.716	5.66	0.286	9.83	0.066	1.52
5,000	5.041	2.94	0.514	1.80	0.260	3.94
10,000	8.859	1.76	1.131	2.20	0.573	2.20
100,000	118.989	13.43	10.014	8.85	4.800	8.38

From 10,000 to 100,000, O(n) predicts a ratio of about 10 and O(n log n) about 12.5. Task A (13.4) and Task C (8.4) match their expected growth. Task B (8.9) is a little under its n log n prediction: at these sizes the log factor adds only about 25%, which is within timing noise, and it is far from the 100 that n^2 would give. The larger sizes were added because runs under a millisecond are dominated by timer precision and JIT warm-up, which is why the early ratios jump around.